

Templates : Template/PG-fr.docx | Template/Presentation_BELACEL_V3.pptx

PDF : TP_Informatique_ENS_1ère_année_AN_03.pdf

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS — Year 1 — English Department

Duration: 1h30

Academic Year: 2025-2026

Learning Objectives

At the end of this lab, the student is able to:

- Master the for loop with range()
- Master the while loop with stop condition
- Use break and continue to control execution
- Implement iterative algorithms (sum, sequence, game)
- Validate repeated inputs with attempt counter

Lab Plan

Phase	Content	Duration
Welcome	Review of loops	10 min
Exercise 1	Multiplication table	15 min
Exercise 2	Sum of the first N integers	15 min
Exercise 3	Fibonacci sequence	20 min
Exercise 4	Number guessing game	20 min
Exercise 5	Repeated password validation (×3)	10 min

Prerequisites

- Labs 01 and 02 completed (variables, conditions, try/except)
- Concept of range() and loops covered in class

Exercise 1 — Multiplication table (15 min)

Problem statement: Create a script `tables.py` that displays the multiplication table of a given number, from 1 to 10, using a for loop.

Expected output:

```
Number : 7
7 × 1 = 7
7 × 2 = 14
...
```

$$7 \times 10 = 70$$

Solution:

```
try:
    n = int(input("Number : "))
    for i in range(1, 11):
        print(f"{n} x {i} = {n * i}")
    except ValueError:
        print("Please enter an integer.")
```

Diagram of a for loop:

```
flowchart TD
    A[Start] --> B[Define n]
    B --> C["Initialize i = 1"]
    C --> D{"i ≤ 10 ?"}
    D -->|Yes| E["Display n × i"]
    E --> F["Increment i (i + 1)"]
    F --> D
    D -->|No| G[End]
```

Exercise 2 — Sum of the first N integers (15 min)

Problem statement: Create a script `somme_n.py` that:

1. Asks the user for an integer N
2. Calculates the sum of integers from 1 to N using a for loop
3. Calculates the sum of even integers from 1 to N
4. Verifies the result with the mathematical formula $S = N(N+1)/2$

Expected output:

```
N = 10
Sum 1 to 10 : 55
Sum of evens : 30
Formula : 55 – Verified ✓
```

Solution:

```
try:
    N = int(input("N = "))
    total = 0
    sum_even = 0
    for i in range(1, N + 1):
        total += i
        if i % 2 == 0:
            sum_even += i
    formula = N * (N + 1) // 2
    print(f"Sum 1 to {N} : {total}")
    print(f"Sum of evens : {sum_even}")
    print(f"Formula : {formula}", end=" – ")
    print("Verified ✓" if total == formula else "Error x")
    except ValueError:
        print("Please enter an integer.")
```

Exercise 3 — Fibonacci sequence (20 min)

Problem statement: Create a script `fibonacci.py` that displays the first N terms of the Fibonacci sequence (N entered by the user). $F(0) = 0$, $F(1) = 1$, $F(n) = F(n-1) + F(n-2)$.

Expected output (n = 10):

```
Number of terms : 10
0, 1, 1, 2, 3, 5, 8, 13, 21, 34
```

Solution:

```
try:
    n = int(input("Number of terms : "))
    a, b = 0, 1
    for _ in range(n):
        print(a, end=" ")
        a, b = b, a + b
    print()
except ValueError:
    print("Please enter an integer.")
```

Exercise 4 — Number guessing game (20 min)

Problem statement: Create a script `devine.py` that:

1. Picks a random number between 1 and 100
2. Asks the user to guess
3. Indicates whether the number is too small or too large
4. Limits to a maximum of 7 attempts
5. Displays the secret number if lost

Expected output:

```
Guess the number between 1 and 100 (7 attempts max).
Your guess : 50
Too high.
Your guess : 25
Too low.
...
```

Flowchart of the guessing game:

```
flowchart TD
    A[Start] --> B["Generate secret number\n(1 to 100)"]
    B --> C["Initialize attempts = 0"]
    C --> D{"attempts < 7 ?"}
    D -->|Yes| E[Enter guess]
    E --> F[Inc. attempts]
    F --> G{"Guess = secret ?"}
    G -->|Yes| H[Display Bravo!]
    G -->|No| I{"Guess < secret ?"}
    I -->|Yes| J[Display Too low]
    I -->|No| K[Display Too high]
    J --> D
    K --> D
    D -->|No| L[Display Lost + secret]
    L --> M[End]
    H --> M
    J --> M
    K --> M
```

Solution:

```
import random

secret = random.randint(1, 100)
attempts = 0
max_attempts = 7

print("Guess the number between 1 and 100 (7 attempts max).")

while attempts < max_attempts:
    try:
        guess = int(input("Your guess : "))
        attempts += 1
        if guess == secret:
            print(f"Bravo! Found in {attempts} attempt(s).")
            break
        elif guess < secret:
            print("Too low.")
        else:
            print("Too high.")
    except ValueError:
        print("Enter a valid number.")

if attempts == max_attempts:
    print(f"Lost! The number was {secret}.")
```

Exercise 5 — Repeated password validation (×3) (10 min)

Problem statement: Create a script `mdp_repet.py` that:

1. Defines a fixed secret password (e.g., "ENS2026")
2. Asks the user to enter the password
3. Grants 3 attempts maximum
4. Displays "Access granted" or "Account blocked"

Expected output:

```
Password : python
Incorrect password (2 attempt(s) remaining)
Password : 1234
Incorrect password (1 attempt(s) remaining)
Password : ENS2026
Access granted ✓
```

Solution:

```
password = "ENS2026"
attempts = 3

while attempts > 0:
    entry = input("Password : ")
    if entry == password:
        print("Access granted ✓")
        break
    else:
        attempts -= 1
        if attempts > 0:
            print(f"Incorrect password ({attempts} attempt(s) remaining)")
        else:
            print("Account blocked – too many attempts.")
```

Diagram of a while loop:

```

flowchart TD
    A[Start] --> B["Initialize attempts = 3"]
    B --> C{"attempts > 0 ?"}
    C -->|Yes| D[Enter password]
    D --> E{"Correct?"}
    E -->|Yes| F[Access granted]
    F --> G[End]
    E -->|No| H[Decrement attempts]
    H --> I{"attempts > 0 ?"}
    I -->|Yes| J[Display remaining attempts]
    J --> C
    I -->|No| K[Account blocked]
    K --> G

```

Self-assessment quiz

1. How many times does the loop for `i in range(5)` : execute?

- a) 4 times
- b) 5 times ✓
- c) 6 times

2. What does the `break` statement do inside a `while` loop?

- a) It skips an iteration
- b) It stops the loop immediately ✓
- c) It restarts the loop

3. What is the sum of integers from 1 to 100 using the formula?

- a) 4950
- b) 5050 ✓
- c) 5150

4. In the Fibonacci sequence, what is $F(5)$ if $F(0)=0$ and $F(1)=1$?

- a) 3
- b) 5 ✓
- c) 8

5. What condition stops a `while True` : loop?

- a) `continue`
- b) `break` ✓
- c) `pass`

Grading Rubric Lab 03

Criterion	Weight	Not
<code>for</code> loop and <code>range()</code>	20%	Do
<code>while</code> loop and condition	20%	Do
<code>break</code> and flow control	20%	Do

Iterative algorithms	20%	Non
Attempt management	20%	No r

Session schedule — 1h30 (90 min)

Phase	Duration	Cumulative total
Welcome and loops review	10 min	10 min
Exercise 1 — Multiplication table	15 min	25 min
Exercise 2 — Sum of first N	15 min	40 min
Exercise 3 — Fibonacci sequence	20 min	60 min
Exercise 4 — Number guessing game	20 min	80 min
Exercise 5 — Repeated validation	10 min	90 min

Pedagogical Note ENS

Teaching transposition

This lab on loops and iterations is crucial for developing the algorithmic thinking of future teachers. Each exercise illustrates a distinct use of repetitive structures. The multiplication table (ex. 1) is the simplest example to introduce for and can be presented as early as 8th grade. The sum of the first N integers (ex. 2) allows a fruitful back-and-forth between algorithmics and mathematics (closed formula vs iterative calculation). The Fibonacci sequence (ex. 3) is an interdisciplinary classic (mathematics, biology, architecture). The number guessing game (ex. 5) introduces data validation (try/except) and random generation (random).

Trainer advice

Clearly distinguish between `for` (when the number of iterations is known) and `while` (when waiting for a condition). Show Thonny's step-by-step debugger to visualize the evolution of variables at each iteration. For middle school students, use exclusively `for` with `range()` and concrete examples (counting the number of vowels in a sentence).

Extension

Implement a "Right Price" game where the computer chooses a random number between 1 and 100 and the user must guess it in a minimum number of attempts. Add an attempt history.